

OS × GPU Vendor Tooling Map

The fine-tuning and serving environment needs a different tool chain depending on OS and GPU vendor. This page visualizes the decision logic and full matrix from TOOLING_MATRIX.md.

DECISION TREE

1

What's the GPU vendor?

Determines the fine-tuning framework — dependent on CUDA (Unsloth, Axolotl), on ROCm, or vendor-agnostic/Metal (MLX).

2

Is the memory architecture dedicated or unified?

With dedicated VRAM (NVIDIA/AMD), the card class sets a fixed ceiling. With unified memory (Apple Silicon), the ceiling is total system RAM, but it can't be upgraded later — the hardware decision is irreversible here.

3

Is the serving target single-user or production?

`llama.cpp` is lightweight and fits a single-user scenario; `vLLM` / `TGI` are built for batch inference and high throughput.

4

Is the quant format compatible with the serving tool?

`GGUF` → the `llama.cpp` ecosystem; `AWQ` / `GPTQ` integrate more natively with `vLLM` / `TGI`. A mismatch forces an extra conversion step (as happened in this project with Qwen3 + the MLX GGUF exporter).

MATRIX

	LINUX	WINDOWS	MACOS
NVIDIA CUDA · dedicated VRAM	RESEARCH Unsloth, Axolotl, LLaMA-Factory + vLLM / TGI / llama.cpp serving GGUF · AWQ · GPTQ	RESEARCH Unsloth (WSL2 önerilir), LLaMA-Factory + llama.cpp / vLLM (WSL2) GGUF · AWQ	– not available in hardware
AMD ROCm · dedicated VRAM	RESEARCH Axolotl (ROCm backend), LLaMA-Factory (kısmi) + llama.cpp (HIP) / vLLM (ROCm) GGUF	RESEARCH Limited — mostly falls back to the Linux path via WSL2 GGUF	– not available in hardware
Apple Silicon Metal/MLX · unified memory	– not available in hardware	– not available in hardware	VERIFIED · THIS PROJECT MLX (mlx-lm) fine-tuning + llama.cpp (Metal backend) serving GGUF → Q4_K_M
CPU-only Intel iGPU · inference only	RESEARCH llama.cpp (CPU) — fine-tuning theoretically possible (Axolotl) but not practical GGUF	RESEARCH llama.cpp (CPU) GGUF	RESEARCH llama.cpp (CPU, without Metal) GGUF

Verified — tested by hand

Research stage — based on ecosystem knowledge

Not available in hardware

TOOL CHOICE NOTES — WHEN A CELL HAS MORE THAN ONE OPTION

Unsloth vs Axolotl vs LLaMA-Factory LINUX/WINDOWS + NVIDIA

Unsloth — first choice when speed and low VRAM on a single GPU are the priority. Claims up to 2x speed and 50-80% less VRAM via custom Triton kernels; weak at multi-GPU distributed training.

Axolotl — when you need multi-GPU / DeepSpeed / FSDP, or want to manage many different model architectures through a single YAML config flow.

LLaMA-Factory — for fast experimentation/prototyping through a GUI, without writing code.

Context for this project: in a single-GPU + single-model scenario, Unsloth would have been the first choice. The move to MLX happened only because Apple Silicon has no CUDA/Triton — Unsloth itself is still one of the fastest options on CUDA, not abandoned, just unusable on this hardware.

vLLM vs TGI vs llama.cpp SERVING, NVIDIA/AMD

vLLM — production serving with many users and high request volume. Stands out on batch throughput via PagedAttention.

TGI — when you're tightly tied to the Hugging Face ecosystem (Inference Endpoints, Transformers compatibility).

llama.cpp — single user / low traffic, light footprint, easy to deploy. GGUF's portability is also a plus — this is why it was chosen for this project's serving chain too.

Why Unsloth isn't in the Linux + AMD cell ROCm LIMITATION

Unsloth's core acceleration is written against CUDA/Triton kernels; there's no official ROCm support — the same reason it was seen not to work on Apple Silicon/Metal during this project. Community forks attempt to port it to ROCm, but it's not stable/official. For production on ROCm, relying on **Axolotl**'s ROCm backend is safer.

WSL2 vs native Windows WINDOWS + NVIDIA

If you're only running serving/inference (e.g. `llama-server`), native Windows is enough

and doesn't add extra complexity. For fine-tuning, moving to WSL2 saves time — Unsloth/Axolotl's setup steps, tested on Linux, have much less friction than on native Windows.

UNIFIED MEMORY — THERE'S NO "VRAM" ON APPLE SILICON

Why this distinction matters

NVIDIA / AMD — Dedicated VRAM

The GPU has its own memory, separate from the CPU (e.g. RTX 4090 → 24GB). If the model doesn't fit in VRAM, offload/swap is needed. To get more, you swap the card.

Apple Silicon — Unified Memory

CPU and GPU share the same physical RAM pool. The fit ceiling is total system RAM — no separate GPU bottleneck, but there's contention with the CPU for memory bandwidth.

Irreversible decision: RAM is soldered on Apple Silicon — it can't be upgraded later. The RAM amount chosen when buying the Mac becomes the final ceiling for GPU memory too. On NVIDIA/AMD you swap the card if VRAM falls short; on Apple, that path doesn't exist.

Example from this project: `Josiefied-Qwen3-4B-abliterated-v1-4bit` (2.26GB) runs comfortably on a 16GB Mac. Quantization (4-bit / Q4_K_M) stays critical under unified memory too — the constraint is no longer "fitting into VRAM" but sharing RAM with the OS and other apps.

tooling_matrix.md — docs/mlx-finetune-test